

DNS Interno al Cluster (CoreDNS)

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Spiegare il ruolo di CoreDNS come componente DNS interno di OpenShift e dove viene deployato
- Descrivere lo schema completo di risoluzione dei nomi: Service, Pod, e search domains
- Configurare stub zones e forwarding per integrare CoreDNS con un DNS aziendale esterno
- Utilizzare `nslookup`, `dig` e `getent hosts` per diagnosticare problemi DNS dall'interno di un Pod
- Descrivere l'integrazione di CoreDNS con Azure Private DNS Zone su ARO

Teoria e Concetti Chiave

CoreDNS in OpenShift

OpenShift usa **CoreDNS** come server DNS interno al cluster. CoreDNS è deployato nel namespace `openshift-dns` e gestito dall'**DNS Operator** tramite la CRD `dns.operator.openshift.io`.

Componenti:

Componente	Namespace	Descrizione
<code>dns-default</code> DaemonSet	<code>openshift-dns</code>	Pod CoreDNS su ogni nodo (uno per nodo)
<code>dns-default</code> Service	<code>openshift-dns</code>	ClusterIP del DNS (es. <code>172.30.0.10</code>)
DNS Operator	<code>openshift-dns-operator</code>	Gestisce la configurazione di CoreDNS
<code>dns.operator.openshift.io</code>	Cluster-scoped	CRD che configura CoreDNS

💡 **Suggerimento:** CoreDNS viene deployato come DaemonSet (un Pod per nodo) per ridurre la latenza DNS: ogni Pod risolve i nomi tramite il CoreDNS locale al proprio nodo.

Come i Pod Risolvono i Nomi

Ogni Pod riceve un file `/etc/resolv.conf` generato automaticamente da Kubelet:

```
nameserver 172.30.0.10          ← IP del Service CoreDNS
search myproject.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

search domains: permettono la risoluzione abbreviata del nome:

- `curl http://webapp` → prova `webapp.myproject.svc.cluster.local` → trovato!
- `curl http://webapp.otherproject` → prova `webapp.otherproject.svc.cluster.local` → trovato!

ndots:5: se il nome ha meno di 5 punti (dots), viene prima provata la risoluzione con i search domains prima di fare una query DNS assoluta.

Schema di Risoluzione dei Nomi

Formato FQDN per i Service

```
<service-name>.<namespace>.svc.cluster.local
```

Esempi:

webapp.myproject.svc.cluster.local	→ ClusterIP del Service "webapp"
mysql.database-ns.svc.cluster.local	→ ClusterIP del Service "mysql" in "database-ns"
mysql-headless.myns.svc.cluster.local	→ lista IP dei Pod (Headless Service)

Formato FQDN per i Pod

```
<pod-ip-trattini>.<namespace>.pod.cluster.local
```

Esempi:

10-128-4-5.myproject.pod.cluster.local	→ Pod con IP 10.128.4.5
--	-------------------------

 **Suggerimento:** Il DNS dei Pod si usa raramente in pratica. Il pattern corretto è sempre referenziare i Service (che forniscono una VIP stabile), non gli IP dei Pod (che cambiano ad ogni riavvio).

Risoluzione Abbreviata (con search domains)

All'interno del namespace **myproject**, un Pod può usare forme abbreviate:

Query abbreviata	Risolta come	Risultato
webapp	webapp.myproject.svc.cluster.local	ClusterIP webapp
webapp.myproject	webapp.myproject.svc.cluster.local	ClusterIP webapp
mysql.database-ns	mysql.database-ns.svc.cluster.local	ClusterIP mysql
api.example.com	api.example.com (FQDN esterno)	IP pubblico

Configurazione CoreDNS: Corefile

Il file di configurazione di CoreDNS è il **Corefile**, gestito come ConfigMap:

```
oc get configmap dns-default -n openshift-dns -o yaml
```

Struttura tipica del Corefile:

```
.:5353 {
  errors
  health {
    lameduck 20s
  }
  ready
  kubernetes cluster.local in-addr.arpa ip6.arpa {
    pods insecure
    fallthrough in-addr.arpa ip6.arpa
  }
  prometheus :9153
  forward . /etc/resolv.conf {    ← forwarding delle query non-cluster al DNS
del nodo
    policy sequential
  }
  cache 900
  reload
  loadbalance
}
```

Stub Zones e Forwarding

Stub zone: configurazione che dice a CoreDNS di inviare le query per un dominio specifico a un server DNS diverso.

Use case: integrare un DNS aziendale on-premise per risolvere hostname interni come `db.corp.example.com`.

Configurazione tramite DNS Operator:

```
# Aggiungere una stub zone per il dominio aziendale
apiVersion: operator.openshift.io/v1
kind: DNS
metadata:
  name: default
spec:
  servers:
  - name: corp-dns
```

```
zones:
- corp.example.com      # ← resolve questo dominio tramite il DNS
aziendale
forwardPlugin:
  upstreams:
    - 192.168.1.53      # ← IP del DNS aziendale on-premise
    - 192.168.2.53      # ← DNS secondario (failover)
```

DNS Operator

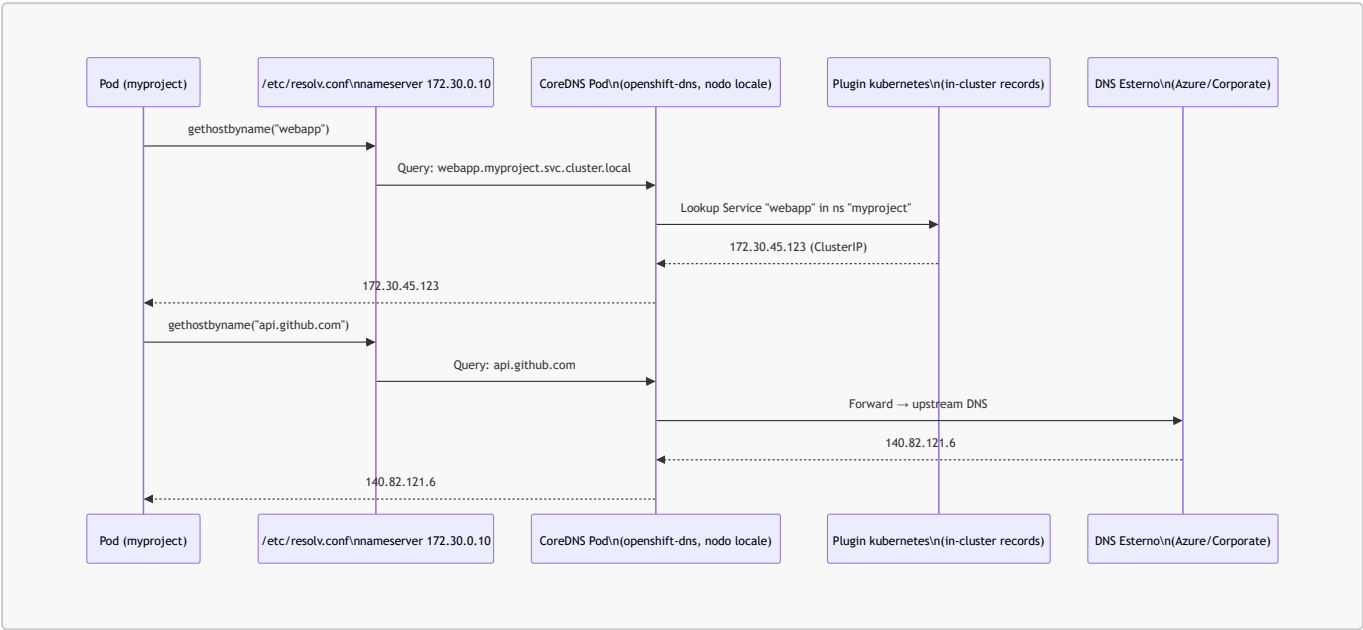
Il **DNS Operator** in OCP gestisce il ciclo di vita di CoreDNS. Configura automaticamente:

- Il ConfigMap `dns-default` con il Corefile
- Il DaemonSet `dns-default` con i Pod CoreDNS
- Il Service `dns-default` con il ClusterIP del DNS

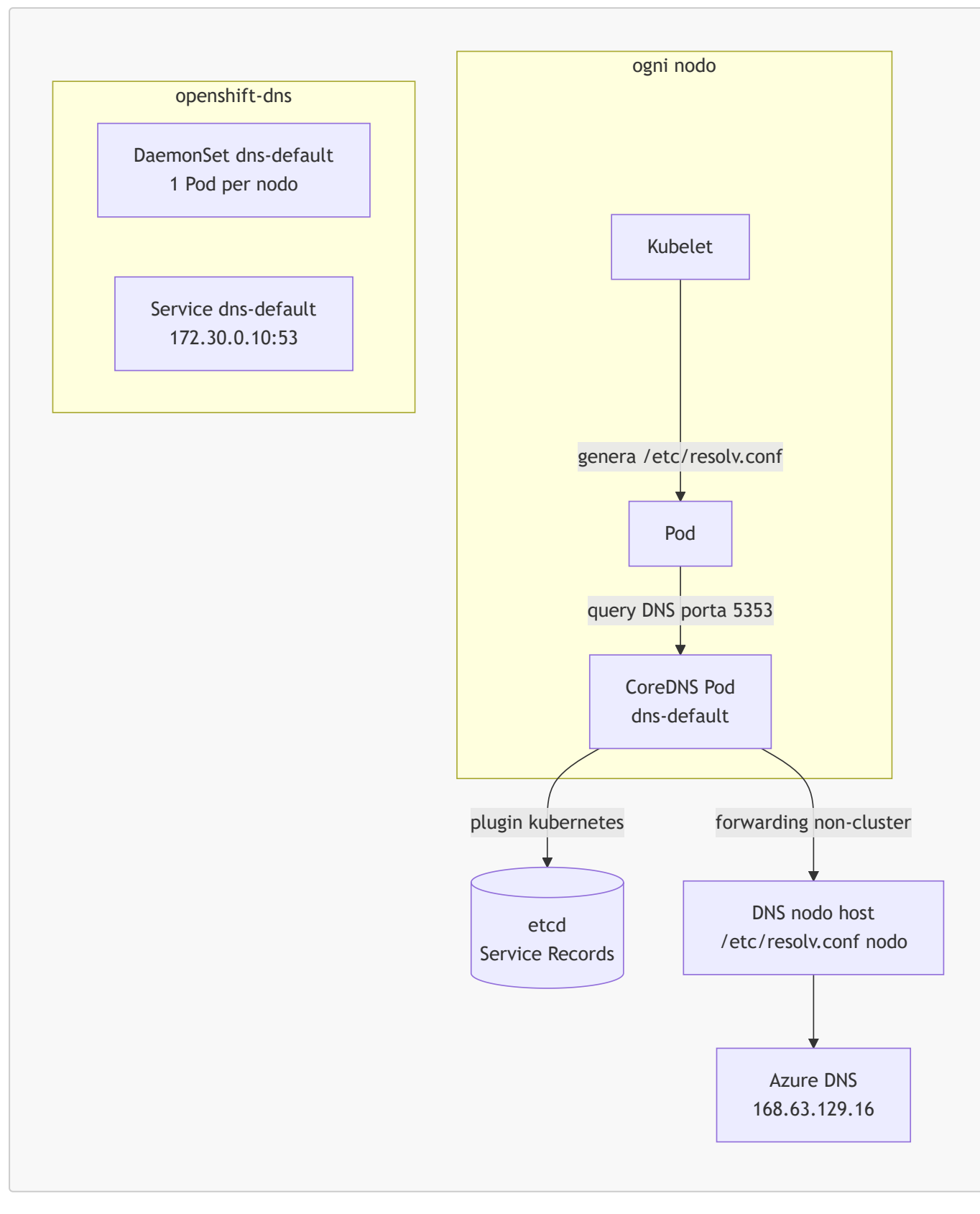
```
# CRD del DNS Operator
oc get dns.operator.openshift.io default -o yaml
```

Diagrammi

Flusso Risoluzione DNS da Pod a Service



Architettura CoreDNS in OCP



Comandi Pratici con Output Atteso

```
# Visualizzare i Pod CoreDNS (DaemonSet, uno per nodo)
oc get pods -n openshift-dns
```

Output atteso:

# NAME	READY	STATUS	RESTARTS	AGE
# dns-default-4fk1m	2/2	Running	0	5d
# dns-default-7nrst	2/2	Running	0	5d
# dns-default-9xyzq	2/2	Running	0	5d

```
# Visualizzare il ClusterIP del Service CoreDNS
oc get svc dns-default -n openshift-dns
```

Output atteso:

# NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
# dns-default	ClusterIP	172.30.0.10	<none>	53/UDP,53/TCP,9153/TCP

5d

```
# Visualizzare la configurazione CoreDNS (Corefile)
oc get configmap dns-default -n openshift-dns -o jsonpath='{.data.Corefile}'
```

Output atteso (estratto):

```
# .:5353 {
#   errors
#   health {
#     lameduck 20s
#   }
#   kubernetes cluster.local in-addr.arpa ip6.arpa {
#     pods insecure
#     fallthrough in-addr.arpa ip6.arpa
#   }
#   forward . /etc/resolv.conf {
#     policy sequential
#   }
#   cache 900
# }
```

```
# Risolvere un Service dall'interno di un Pod (nslookup)
oc exec -it debug-pod -n myproject -- nslookup webapp
```

Output atteso:

```
# Server:      172.30.0.10
# Address:     172.30.0.10#53
```

```
#  
# Name:    webapp.myproject.svc.cluster.local  
# Address: 172.30.45.123
```

```
# Risolvere un Service in un altro namespace (FQDN)  
oc exec -it debug-pod -n myproject -- nslookup mysql.database-ns.svc.cluster.local  
  
# Output atteso:  
# Name:    mysql.database-ns.svc.cluster.local  
# Address: 172.30.67.89
```

```
# Verificare il /etc/resolv.conf generato nel Pod  
oc exec -it debug-pod -n myproject -- cat /etc/resolv.conf  
  
# Output atteso:  
# search myproject.svc.cluster.local svc.cluster.local cluster.local  
# nameserver 172.30.0.10  
# options ndots:5
```

```
# Usare dig per una risoluzione DNS completa con RTT  
oc exec -it debug-pod -n myproject -- dig +short  
webapp.myproject.svc.cluster.local  
  
# Output atteso:  
# 172.30.45.123
```

```
# Visualizzare la configurazione del DNS Operator  
oc get dns.operator.openshift.io default -o yaml  
  
# Output atteso (estratto):  
# spec:  
#   nodePlacement: {}  
# status:  
#   clusterDomain: cluster.local  
#   clusterIP: 172.30.0.10  
#   conditions:  
#     - type: Available  
#       status: "True"
```

```
# Diagnosticare problemi DNS: log dei Pod CoreDNS  
oc logs -n openshift-dns dns-default-4fklm -c dns --tail=20
```

```
# Output atteso:  
# [INFO] 10.128.4.5:54321 - 12345 "A IN webapp.myproject.svc.cluster.local. udp 58  
false 512" NOERROR qr,aa,rd 103 0.000234s
```

Esempi YAML Completi

Configurazione Stub Zone per DNS Aziendale

```

apiVersion: operator.openshift.io/v1
kind: DNS
metadata:
  name: default # ← nome fisso, è un singleton
spec:
  servers:
    # Stub zone 1: dominio aziendale on-premise
    - name: corp-internal-dns # ← nome identificativo (unico)
      zones:
        - corp.example.com # ← tutte le query per *.corp.example.com
        - internal.example.com # ← anche questo sottodominio
      forwardPlugin:
        policy: Sequential # ← Sequential o Random tra gli upstreams
        upstreams:
          - 192.168.1.53 # ← DNS primario aziendale
          - 192.168.2.53 # ← DNS secondario aziendale (failover)

    # Stub zone 2: Azure Private DNS Zone personalizzata
    - name: azure-private-zone
      zones:
        - privatelink.database.windows.net # ← Azure SQL private endpoint DNS
      forwardPlugin:
        upstreams:
          - 168.63.129.16 # ← Azure DNS resolver (magic IP Azure)

    # Configurazione opzionale del resolver upstream predefinito
  upstreamResolvers:
    upstreams:
      - type: SystemResolvConf # ← usa il /etc/resolv.conf del nodo (default)
    policy: Sequential
    protocolStrategy: ""

```

Pod con dnsConfig Personalizzato

```

apiVersion: v1
kind: Pod
metadata:
  name: custom-dns-pod
  namespace: myproject
spec:
  # dnsPolicy definisce come il Pod risolve i nomi
  # ClusterFirst (default): usa CoreDNS per nomi cluster, forwarda il resto
  # None: usa solo dnsConfig personalizzato
  dnsPolicy: ClusterFirst
  dnsConfig:

```

```
# Aggiunge search domains personalizzati oltre a quelli predefiniti
searches:
- corp.example.com          # ← "db" viene provato come "db.corp.example.com"
- internal.example.com
# Aggiunge opzioni resolv.conf personalizzate
options:
- name: ndots
  value: "3"                 # ← riduce ndots per evitare troppe query con
search
- name: timeout
  value: "2"                 # ← timeout per ogni query DNS in secondi
containers:
- name: app
  image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
resources:
  requests:
    memory: "64Mi"
    cpu: "50m"
  limits:
    memory: "128Mi"
    cpu: "200m"
```

Note Specifiche per Azure ARO

Azure DNS e CoreDNS su ARO

Su ARO, i nodi usano come DNS upstream il **Azure DNS resolver** (**168.63.129.16**), un IP magico di Azure che:

- Risolve i nomi DNS pubblici di Azure (es. ***.blob.core.windows.net**)
- Risolve le **Azure Private DNS Zone** configurate nel VNet

```
Pod → CoreDNS (openshift-dns) → forwarding → DNS nodo → Azure DNS 168.63.129.16
```

Azure Private DNS Zone per Private Endpoints

Per accedere a risorse Azure con Private Endpoint (es. Azure SQL, Azure Storage) da un cluster ARO, è necessario che CoreDNS risolva correttamente i nomi **privatelink.***:

```
# Verificare che la Private DNS Zone sia linkata alla VNet ARO
az network private-dns zone list \
  --resource-group myResourceGroup \
  --output table

# Verificare che il VNet link esista
az network private-dns link vnet list \
  --resource-group myResourceGroup \
  --zone-name "privatelink.database.windows.net" \
  --output table
```

Se la Private DNS Zone è linkata alla VNet ARO, la risoluzione funzionerà automaticamente tramite Azure DNS **168.63.129.16** senza configurazione aggiuntiva di CoreDNS.

Configurazione DNS per Cluster Privati ARO

Per cluster ARO privati (senza accesso internet), è possibile che il DNS di default di Azure non risolva domini internet. In questo caso:

```
# Configurare un forwarder custom nel DNS Operator
oc edit dns.operator.openshift.io default

# Aggiungere:
# spec:
#   upstreamResolvers:
#     upstreams:
#       - type: Network
#         address: 192.168.1.53    ← DNS aziendale con accesso a internet
#         port: 53
```

⚠ Attenzione: Su ARO, non modificare il ConfigMap `dns-default` direttamente: le modifiche verranno sovrascritte dall'DNS Operator. Usare sempre la CRD `dns.operator.openshift.io/default` per la configurazione.

Riepilogo dei Punti Chiave

- CoreDNS è il DNS interno di OCP, deployato come DaemonSet in `openshift-dns` (un Pod per nodo per minimizzare la latenza)
 - Ogni Pod riceve `/etc/resolv.conf` con il ClusterIP di CoreDNS come nameserver e i search domains del namespace
 - Il FQDN di un Service è `<service>.<namespace>.svc.cluster.local`; le forme abbreviate funzionano grazie ai search domains
 - Le stub zones nel DNS Operator permettono di delegare domini specifici a server DNS aziendali esterni
 - `nslookup`, `dig` e `getent hosts` sono i comandi per diagnosticare problemi DNS dall'interno di un Pod
 - Su ARO, Azure DNS (`168.63.129.16`) è l'upstream predefinito; le Azure Private DNS Zone link alla VNet vengono risolte automaticamente
 - Non modificare il ConfigMap `dns-default` direttamente: usare la CRD `dns.operator.openshift.io/default`
-

Domande di Verifica

1. Un Pod nel namespace `myapp` vuole connettersi al Service `mysql` nel namespace `database`. Quale hostname usa?

- A) `mysql.svc.cluster.local`
- B) `mysql.database`
- C) `mysql.database.svc.cluster.local` ✓
- D) `mysql.database.pod.cluster.local`

2. Un'applicazione su ARO non riesce a risolvere `mydb.privatelink.database.windows.net`. Quale configurazione Azure manca?

- A) Una regola NSG per la porta 53 UDP
- B) Un stub zone nel DNS Operator per `privatelink.database.windows.net`
- C) Il link della Azure Private DNS Zone al VNet ARO ✓
- D) Un record CNAME nella Route OpenShift

3. Quale file viene automaticamente generato da Kubelet in ogni Pod per la configurazione DNS?

- A) `/etc/hosts`
- B) `/etc/resolv.conf` ✓
- C) `/etc/coredns/Corefile`
- D) `/var/run/dns.conf`

4. Un amministratore vuole che i Pod risolvano `*.corp.internal` tramite il DNS aziendale `10.1.1.53`. Come configura CoreDNS?

- A) Modificando direttamente il ConfigMap `dns-default` in `openshift-dns`
- B) Aggiungendo un server stub zone nella CRD `dns.operator.openshift.io/default` ✓
- C) Creando una NetworkPolicy che permette UDP 53 verso `10.1.1.53`
- D) Modificando `/etc/resolv.conf` su ogni nodo worker

5. Quale comando verifica la risoluzione DNS di un Service dall'interno di un Pod?

- A) `oc get dns webapp.myproject.svc.cluster.local`
- B) `oc describe service webapp | grep DNS`
- C) `oc exec -it mypod -- nslookup webapp.myproject.svc.cluster.local` ✓
- D) `oc logs -n openshift-dns dns-default | grep webapp`